BMF System Status and Hardening Register

Prepared: 08 November 2026 **Purpose:** One repository-grounded view of shipped BMF work, open work, and the files that should receive the next strengthening pass. **Source rule:** This register compares `TODO.md`, `AGENTS.md`, route documentation, tests, and the newer ExecPlans. Prompt examples are design references only; no illustrative names, balances, IDs, or addresses are treated as system data.

Executive summary

NaviServer already has working customer and admin portals, loan and payment workflows, role/store authorization, OTP, audit/security features, reports, backups, health monitoring, and both JSON and SQLite storage paths. The main remaining risks are operational verification, visual browser QA, data migration decisions, input-hardening depth, and stale/conflicting project checklists.

Recent update (08 November 2026): customer-requested custom loan dates now determine exact calendar-day terms. Interest is calculated from a store-scoped policy shared by the customer quote and server request handler. Admin and Franchise Admin can edit the policy; edits are validated and audited. The term begins on approval and the custom extension rate applies only through the due date. Existing grace/default penalties and already-issued loans are unchanged.

This is a source-led status register, not a new security certification. The completion marks below reflect the most recent relevant plan or test record. Browser and public-host checks remain unverified unless explicitly stated.

Completed or implemented

These areas are marked complete by the newer plans and/or the project TODO. Their current behavior should still be protected by regression tests when changed.

Area	Implemented capability	Evidence / primary files
Authentication and onboarding	Login/OTP, password reset, invite-based signup, device-token protected requests, generic reset responses	`controllers/authController.js`, `middleware/security.js`, `middleware/otp.js`, `middleware/phoneNumber.js`, `middleware/shareLinks.js`, `routes/index.js`; `docs/plans/invite-enforcement.md`
Loan lifecycle	Request, review, approval, decline, grace/default handling, actions, audit fields, notifications, scoped authorization	`controllers/loanController.js`, `middleware/loanNotifications.js`, `scheduler.js`; `docs/plans/loan-lifecycle.md`
Tenure and due dates	New loan due dates derive from the approval date in the Lusaka calendar; one week is seven calendar days regardless of request wait time	`controllers/loanController.js`, `public/admin.js`, `test/loanWorkflow.test.js`, `test/storeScope.test.js`; `docs/plans/guardian-security-profile-cron.md`
Custom-date tenure and rates	Customer-selected dates are priced as exact days; the 1-4 week tiers are preserved, custom terms interpolate through 28 days and use a configurable daily extension after that. Admin and Franchise Admin can edit validated store-scoped rates; extension interest stops at the due date.	`services/loanTerms.js`, `controllers/loanController.js`, `public/user.html`, `public/admin.html`, `public/admin.js`, `test/loanInterestPolicy.test.js`; `docs/plans/custom-loan-tenure-interest.md`

Area	Implemented capability	Evidence / primary files
Payment controls	Customer proof submissions remain pending until in-scope staff review; approved payments are recorded once in the ledger with receipt and balance fields	`controllers/loanController.js`, `controllers/fileController.js`, `public/user.html`, `public/admin.js`, `test/verifiedPayments.test.js`, `test/userPortalPayments.test.js`
Customer portal and profile	Loan history, loan details, payment history/proof flow, inquiries, profile summaries, ownership-checked attachments	`public/user.html`, `public/dashboard.html`, `public/dashboard.js`, `controllers/fileController.js`; `docs/plans/admin-customer-portal-and-verified-payments.md`
Admin pages and customer records	Admin directories, customer profile, Quick Screen details, table overflow handling, report entry points, scoped actions and status views	`public/admin.html`, `public/admin.js`, `public/style.css`; `docs/plans/admin-directory-access-refresh.md`, `docs/plans/guardian-security-profile-cron.md`
Security permissions	Fixed Franchise Admin role/group, custom permission groups, OTP-backed access approvals, store-scoped decisions, server-side feature gates, premium-data controls	`controllers/securityController.js`, `controllers/accessApprovalController.js`, `middleware/featurePermissions.js`, `middleware/premiumPrivacy.js`, `routes/index.js`, `public/admin.js`; `test/securityPermissionGroups.test.js`, `test/accessApprovalController.test.js`
Audit and alerts	Append-only redacted audit records, integrity checks, threat alerts, breakage alerts with deduplication/fallback queue	`controllers/auditController.js`, `middleware/auditLog.js`, `middleware/errorAudit.js`, `middleware/logger.js`; `docs/plans/audit-guardian.md`, `docs/plans/breakage-alerts.md`
Reports	Typed reports, Excel/PDF exports, borrower statements, date/filter controls, daily scheduler and report tracking	`controllers/reportController.js`, `controllers/reportSpec.js`, `controllers/reportTracking.js`, `services/bmfReports.js`, `services/bmfExcel.js`, `services/bmfPdf.js`, `public/admin.js`; `docs/plans/report-system-refresh.md`
Backups and storage	Backup/restore, SQLite document-store path, import tooling, JSON mirror/fallback; SQLite cutover intentionally remains a separate operational decision	`controllers/backupController.js`, `middleware/db.js`, `middleware/dataStore.js`, `scripts/import-json-to-sqlite.js`; `docs/plans/backup-restore.md`, `docs/plans/core-organs.md`
Monitoring and scheduled jobs	Health endpoints, startup checks, health heartbeat, cleanup sweeps, overdue/report notifications, cron callbacks wrapped for asynchronous execution, HTTPS keep-alive	`controllers/statusController.js`, `middleware/startupHealth.js`, `middleware/healthWatch.js`, `middleware/kidneys.js`, `middleware/dailyReport.js`, `scheduler.js`, `server.js`; `docs/plans/guardian-security-profile-cron.md`
Access boundaries	Route map, store scoping, signed upload URLs, private-network vault/shutdown controls, protected data responses	`routes/index.js`, `routes/apply.js`, `routes/users.js`, `middleware/storeScope.js`, `middleware/signedUrl.js`, `middleware/localControl.js`, `docs/ROUTES.md`; `docs/plans/public-access-and-consistency-audit.md`

Remaining work — priority order

P0 — prove the production boundary before public launch

- [ ] Complete the route × identity authorization matrix for anonymous, customer, store admin, franchise admin, and admin identities. Include method, status, store boundary, and response-field expectations. `test/routeMatrix.test.js` is named as missing in an older plan; reconcile that note with the current route-wiring and role tests.
- [ ] Verify the configured HTTPS public URL from a separate device: signup/login, static assets, protected API denial, role/store denial, upload signing, and signed-link expiry.
- [ ] Verify the production deployment settings without recording secrets in this document: required signing/master keys, `PUBLIC_DEPLOY`, trusted proxy count, HTTPS termination, and email delivery configuration.
- [ ] Add a release gate covering environment checks, data integrity, auth/OTP, invite flow, loan request/approval/payment, report exports, backup/restore, and scheduler health.
- [ ] Decide the Windows service/Task Scheduler account and permissions; verify scheduled Lusaka-time jobs run when no user is logged in. Do not enable a background service until ownership and recovery behavior are documented.

P1 — finish user-facing validation and known follow-ups

- [ ] Run authenticated desktop and mobile browser QA for customer, admin, HR, Quick Screen, customer profile, loan details, payment review, and report pages. Current plans record code/test checks but not a completed visual review.
- [ ] Visually verify the latest admin light-mode palette across every admin page and dialog. The CSS override was added, but rendered browser verification has not been recorded.
- [ ] Reconcile modal close behavior: the customer/admin portal plan still has an unchecked close-button/backdrop/Escape task, while the newer guardian plan says this behavior was fixed. Verify all dialogs and update both records to the same result.
- [ ] Verify admin detail-page Back buttons and Quick Screen applicant attachment links in an authenticated session; this remains unchecked in `docs/plans/admin-directory-access-refresh.md`.
- [ ] Complete end-to-end customer flow: signup → OTP/login → dashboard → request → admin decision → payment proof → approval → customer balance/history. Exercise both success and failure feedback.
- [ ] Complete the remaining server-side loan validation review: required fields, allowed tenure/urgency/payment modes, amount boundaries, date validity, duplicate/replay behavior, and derived totals. Do not trust client-computed interest, balance, or due dates.
- [ ] Review the four open loan records identified in the 06 November read-only date audit whose saved due dates differ from approval-date-plus-tenure calculation. No records were changed. Any correction requires an authorized data decision because dates affect grace/default status, reminders, penalties, and customer obligations.
- [ ] Check visual consistency of status badges, summary cards, form errors, empty states, and loading states across both portals.

P2 — data, deployment, and maintenance decisions

- [ ] For SQLite cutover, run a documented `dual`-mode soak, compare JSON and SQLite reads/writes, backup and restore, monitor health/fallback behavior, and only then decide whether to set `DB_MODE=sqlite`. Keep JSON rollback available.
- [ ] Configure public deployment secrets and URL in the deployment environment; do not put secret values in source control or this register.
- [ ] Review scratch artifacts in the repository root and `tools/` (*`_out.txt`*, `analysis.txt`, `checkpoint_`, `check*.txt`, one-off scripts, and temporary controller/report copies). Identify ownership and remove only confirmed obsolete files; do not delete without checking whether they contain needed evidence or customer data.
- [ ] Consolidate stale ExecPlans and TODOs as described below so maintainers have one source of truth.

File-level strengthening map

“Strengthen” here means the listed validation or maintenance work is still appropriate; it does not claim that each file has a known defect.

File set	Strengthening focus
`server.js`, `scheduler.js`, `config/index.js`, `package.json`, `start-server.cmd`, `stop.js`, `cli.js`	Environment validation, proxy assumptions, graceful shutdown, cron timezone/reliability, Windows service/runbook, dependency and deployment checks.
`routes/index.js`, `routes/apply.js`, `routes/users.js`, `docs/ROUTES.md`	Complete route × identity matrix; confirm static-before-dynamic ordering; verify every frontend URL and method maps to the intended gate.
`controllers/authController.js`, `controllers/accessApprovalController.js`, `controllers/securityController.js`, `middleware/security.js`, `middleware/featurePermissions.js`, `middleware/storeScope.js`, `middleware/localControl.js`, `middleware/shareLinks.js`	Recheck role/store boundaries, OTP state and expiry, approval notice recipients, request replay, and public/private path boundaries.
`controllers/loanController.js`, `middleware/loanNotifications.js`, `controllers/fileController.js`, `middleware/signedUrl.js`, `middleware/phoneNumber.js`	Finish input/financial invariants, date edges, duplicate/replay guards, payment-proof ownership, signed-link expiry, and stable customer error messages.
`controllers/reportController.js`, `controllers/reportSpec.js`, `controllers/reportTracking.js`, `services/bmfReports.js`, `services/bmfExcel.js`, `services/bmfPdf.js`, `services/bmfPromptReports.js`, `services/bmfPromptExcel.js`, `middleware/dailyReport.js`	Reconcile old report plan with shipped report refresh; validate live data, store scopes, export privacy, financial formulas, filenames, and scheduler delivery.
`controllers/backupController.js`, `middleware/db.js`, `middleware/dataStore.js`, `scripts/import-json-to-sqlite.js`, `data/*.json`, `data/*.bak`, `data/navi.db*`	Document retention and restore drills, JSON/SQLite parity, atomicity, permissions, backup integrity, and safe database cutover. Keep data contents out of docs/logs.
`controllers/auditController.js`, `middleware/auditLog.js`, `middleware/errorAudit.js`, `middleware/logger.js`, `middleware/healthWatch.js`, `middleware/startupHealth.js`, `middleware/kidneys.js`	Verify audit-chain checks, credential redaction, alert delivery/fallback, health signal coverage, and cleanup behavior. `middleware/logger.js` is protected: do not edit without explicit instruction.
`controllers/feedbackController.js`, `controllers/sessionController.js`, `controllers/statusController.js`, `controllers/logController.js`, `middleware/email.js`, `middleware/otp.js`, `middleware/premiumPrivacy.js`	Verify response privacy, session/device token hygiene, email delivery semantics, premium masking, and service failure behavior. `middleware/otp.js` is protected: preserve its flow unless explicitly instructed.
`public/admin.html`, `public/admin.js`, `public/style.css`	Browser QA for theme consistency, profile/detail dialogs, Quick Screen, forms/tables, loading/error/success feedback, and long IDs/text wrapping. The admin light-mode fix still needs rendered verification.
`public/user.html`, `public/dashboard.html`, `public/dashboard.js`, `public/login.html`, `public/signup.html`, `public/forgot-password.html`, `public/index.html`, `public/quick-screen.html`	Desktop/mobile end-to-end journey, accessible validation/feedback, attachment links, theme parity, safe rendering, and route correctness.
`test/*.test.js`	Keep financial/security behavior covered; add or reconcile the route × identity matrix and browser-level checks. Historical pass counts are not proof of the current working tree.
`TODO.md`, `docs/PLANS.md`, `docs/ROUTES.md`, `docs/Admin-panel.md`, `docs/plans/*.md`, `AGENTS.md`	Remove contradictions, preserve architectural rules, mark superseded tasks, and link all future work to this register or an active ExecPlan.

Protected implementation files

Per `AGENTS.md`, do not modify `middleware/logger.js`, `middleware/otp.js`, or `middleware/dataStore.js` without explicit instruction. Preserve existing controller exports, controller try/catch circuit breakers, `req.authUser`, current role gates, `routes/index.js`

registration, and the difference between `GMAIL_USER` and `ADMIN_EMAILS`.

Documentation conflicts to resolve

1. `TODO.md` previously marked loan request timeframe/urgency, loan history, admin loan review, and some reporting work incomplete. Custom-date pricing and rate management are now recorded complete there; reconcile any remaining stale checkboxes against current tests before relying on the list.
2. `docs/plans/bmf-reports.md` contains an old unchecked build plan. `docs/plans/report-system-refresh.md` records the report system as implemented and documents successful generation. Mark the older plan superseded or update it from current evidence.
3. `docs/plans/public-internet-access.md` says the route matrix is missing and includes an older test-count note. Compare it with current tests and the newer public-access audit before carrying that status forward.
4. Modal-close work is marked complete in the guardian plan and open in the customer portal plan. Verify once and synchronize both.
5. Several plans contain historical test counts or environment notes. Keep them as dated history; use a fresh run and deployment check for release decisions.

Live-data and privacy notes

- This register contains no customer names, account values, receipts, user emails, or secret values.
- The last recorded due-date audit found four open loans with stored dates that differ from the corrected approval-date calculation. The data was left unchanged; use a separately approved, backed-up migration if operations chooses to correct it.
- Do not place `.env`, raw user/payment records, device tokens, NRC documents, or signed upload links in project documentation or diagnostic output.

Suggested next sequence

1. Reconcile stale TODO/ExecPlan entries against the newer implementation plans.
2. Complete the route × identity authorization matrix and deployment/public-URL verification.
3. Perform authenticated desktop/mobile browser QA, including light mode, dialogs, profile, and Quick Screen.
4. Finish loan-validation review and make an explicit decision on the four saved due dates.
5. Establish a release checklist and operational service/SQLite runbooks.